

Toward Real-Time Dynamic 4D Gaussian Splatting on Mobile GPUs: A Survey

Lianghao Zhang
Xiaomi Inc., Graphics Architecture Department
Beijing, China
lianghao@xiaomi.com

ABSTRACT

本综述系统性整理 4D Gaussian Splatting (4DGS) 在移动端 GPU (Snapdragon 8 Gen 4 / Adreno 830 + Vulkan 1.3) 实时渲染方向的最新进展, 涵盖 4DGS 表示路线、训练加速、渲染加速、移动端部署四大主题, 辅以数据集 / 评估指标 / 未来方向的系统讨论。本文与项目 README 互补——前者侧重工程落地路径, 后者侧重学术调研纵深。

1 INTRODUCTION

动态场景的实时神经渲染是计算机图形学近年来的核心研究方向之一。3D Gaussian Splatting (3DGS) [6] 自 2023 年提出以来, 凭借显式点云表示 + 可微分光栅化, 在桌面 GPU 上实现了 1080p 实时辐射场渲染。然而, 将其扩展到动态场景 (4DGS) 时面临三个核心挑战: (1) 时间维度参数化方式选择; (2) 动静态内容分离的存储开销; (3) 移动端 GPU 的算力 / 带宽 / 显存三重约束。

本综述聚焦 移动端实时 4DGS, 系统性梳理 2023–2026 H1 的 50+ 篇代表性工作, 回答以下问题:

- 4DGS 表示路线有哪些? 各自权衡如何?
- 训练阶段 (pruning / quantization / temporal redundancy) 如何压缩?
- 渲染阶段 (光栅化 / ray tracing / mesh extraction) 怎样加速?
- Snapdragon / Adreno 当前瓶颈在哪? 未来路径是什么?

与项目 README 的关系: 本综述与项目主文档 README.md 互补——前者侧重学术调研纵深, 后者侧重 Snapdragon 8 Gen 4 / Adreno 830 + Vulkan 1.3 的工程落地路径。综述面向研究者, README 面向工程师。

本节对应 paper notes 索引: [paper-notes/INDEX.md §A]

2 BACKGROUND

2.1 3D Gaussian Splatting

3DGS [6] 将场景表示为一组各向异性 3D 高斯椭球, 每个高斯由位置 / 协方差 / 不透明度 / 球谐系数参数化。渲染阶段使用可微分光栅化器将高斯投影到屏幕空间, 按 alpha-blending 合成像素颜色。

2.2 4D Gaussian Splatting

4DGS 在 3DGS 基础上引入时间维度, 主流参数化方式分三类:

- **Canonical + Deformation Field:** 4DGS [12], Deformable-3DGS [13], canonical 空间静态表示 + per-frame deformation MLP
- **动静态分离:** 4DGS-1K [14], MEGA [15], STV 评分或 buffer-A/B 残差编码

Table 1: 4DGS 表示路线对比 (5 类)

类别	代表工作	优点	代价
Canonical + Def	4DGS / Deformable-3DGS	灵活	MLP 推理
动静态分离	4DGS-1K / MEGA	静态部分复用	评分误差
Continuous-Time	RetimeGS	帧率自由	时间参数化
Feed-forward	L2D2-GS	消除 per-scene opt	训练数据
Memory-efficient	Sparse4DGS / CubifyGS	显存友好	实现复杂度

- **Continuous-Time:** RetimeGS [11], 消除 temporal aliasing + ghost-free frame interpolation

2.3 Mobile GPU 基础

移动端 GPU (Adreno 830) 受限: 算力 (2 TFLOPs FP32)、显存带宽、Vulkan 1.3 驱动成熟度、tile-based rendering 架构对 alpha-blending 的开销。

本节对应 paper notes 索引: [paper-notes/INDEX.md §A / B]

3 REPRESENTATION TAXONOMY

按时间维度参数化方式, 4DGS 表示路线可分为 5 类。下表给出代表工作与权衡对比。

3.1 Canonical + Deformation Field

4DGS [12] 提出 canonical 空间静态高斯 + 时空 deformation field, 通过 MLP 输出每个高斯在每帧的位移。Deformable-3DGS [13] 引入 canonical anchor 机制, 降低 deformation 搜索空间。

3.2 动静态分离

4DGS-1K [14] (本项目直接对标) 用 STV 评分将高斯分为静态 / 动态, 静态部分跨帧复用。MEGA [15] 用 buffer-A/B 残差编码实现结构化动静态分离。

3.3 Continuous-Time

RetimeGS [11] (CVPR 2026 Oral, HKUST+Netflix) 用 continuous-time 参数化消除 temporal aliasing, 支持 ghost-free frame interpolation。

3.4 Feed-forward

L2D2-GS [10] (小米 + 北大联合, ECCV 2026) 用 feedforward 网络 + 自监督 densification policy, 消除 per-scene optimization, 对作者 heliangliang@xiaomi.com 有合作背景。

3.5 Memory-Efficient

Sparse4DGS / CubifyGS 等工作通过稀疏化、object-level asset 化等方式降低显存占用。

本节对应 paper notes 索引: [paper-notes/INDEX.md §A / B]

4 TRAINING ACCELERATION

训练阶段 (per-scene optimization, 通常 30–60 min) 的加速是移动端部署的前置条件。三类主流方法:

4.1 Temporal Pruning

基于时间冗余的高斯剪枝。OMG4 [7] 提出 minimal 4DGS, 对 imperceptible 时间部分剪枝。SpeeDe3DGS (UMD) 用 temporal pruning + motion compensation 实现 13.71× 加速。

4.2 Compression / Quantization

4DGS-CC [1] 提出 contextual coding framework, PD-4DGS 提出 progressive decomposition + R-DO (Rate-Distortion Optimization)。CAGS 实现色彩自适应的动静态分层 streaming。

4.3 Feed-forward Training-free

L2D2-GS [10] 走 feedforward 路线, 完全消除 per-scene opt。Flux-GS / Mobile-GS 团队的近期工作也在探索 training-free 路径。

本节对应 paper notes 索引: [paper-notes/INDEX.md §B / C]

5 RENDERING ACCELERATION

渲染阶段的加速聚焦三个方向:

5.1 光栅化优化

Mobile-GS [3] (Snapdragon 8 Gen 3, 127 FPS, Vulkan 2.0)、Flux-GS [2] (Snapdragon 8 Gen 3, 147 FPS @ 2.1 MB Indoor) 是当前移动端光栅化的天花板, 均来自 Mobile-GS 团队。

5.2 Ray Tracing 趋势

4DGRT [8] (NTU+Intel) 提出 4DGS Ray Tracing, 代表未来高保真渲染路径。GS-NFS [5] (NVIDIA Research) 在 Jetson Orin mobile GPU 上达到 4DGS 25 FPS decode。

5.3 Mesh Extraction

Lumina [4] (SJTU+Rochester) 走 mobile neural rendering 路线, 4.5× speedup + 5.3× energy efficiency。

5.4 Hardware Acceleration

Neo [9] 提出 Reuse-and-Update Sorting Accelerator, on-device 3DGS 硬件加速。

本节对应 paper notes 索引: [paper-notes/INDEX.md §C / D]

6 MOBILE DEPLOYMENT

6.1 当前现状

Snapdragon 8 Gen 4 + Adreno 830 是 2026 H1 移动端最强 SoC 之一, FP32 算力 2 TFLOPs, Vulkan 1.3 驱动成熟。已部署的 4DGS 工作:

- Mobile-GS / Flux-GS: 桌面 3DGS 实时, 4D 仍在探索

- Lumina: 移动端 neural rendering 标杆
- GS-NFS: Jetson Orin (边缘设备) 4DGS decode

6.2 核心瓶颈

- (1) **显存带宽**: 4DGS 的时空联合高斯数远大于静态 3DGS, bandwidth-limited 严重
- (2) **Vulkan 1.3 驱动成熟度**: compute shader 优化、subgroup operations、tile-based rendering 兼容
- (3) **功耗墙**: 移动端 thermal throttling 导致持续渲染掉帧
- (4) **动静态分离的存储开销**: 即使分离后, 动态部分仍占大头

6.3 本项目方向

本项目 README 锁定的目标是 *Snapdragon 8 Gen 4 / Adreno 830 + Vulkan 1.3* 实时渲染动态 4DGS, 详见项目主文档 [README.md] 与 [01-/02-/03-]。

本节对应 paper notes 索引: [paper-notes/INDEX.md §D / E]

7 DATASETS & METRICS

7.1 常用数据集

4DGS 训练 / 评估常用的动态数据集包括:

- **D-NeRF** (Synthetic): 8 个合成场景, 简单形变
- **DyCheck** (iPhone): 真实动态场景, 移动端采集
- **Nerfies / HyperNeRF**: 单目动态人体 / 物体
- **Plenoptic Video** (Google): 12 相机阵列, 高质量
- **Meet Room / CMU Panoptic**: 多视角多人交互

7.2 评估指标

- **质量**: PSNR / SSIM / LPIPS (per-frame)
- **时序一致性**: Temporal LPIPS / Flow warping error
- **速度**: FPS (decode + render) / 训练时间 (min)
- **存储**: 模型大小 (MB) / 显存峰值 (MB)
- **移动端特有**: 功耗 (mW) / 热稳定性 (sustained FPS)

本节对应 paper notes 索引: [paper-notes/INDEX.md §F]

8 DISCUSSION & FUTURE DIRECTIONS

8.1 未来 5 条方向

- (1) **Feed-forward 4DGS**: 消除 per-scene optimization, L2D2-GS [10] 是开端, 后续可能出现 end-to-end “一段视频进, 4DGS 出” 的模型
- (2) **Hardware-Software Co-design**: Neo [9] 的 Reuse-and-Update Sorting Accelerator 是开端, 移动端 GPU 可能引入专用 4DGS 计算单元
- (3) **4DGS Ray Tracing**: 4DGRT [8] 路线, 高保真场景的最终路径, 但移动端 ray tracing 仍是开放问题
- (4) **Temporal Compression**: 动静态分离 + 残差编码 + 上下文编码三件套可能收敛, 形成 4DGS 压缩标准
- (5) **Cross-Domain**: 4DGS + 物理模拟 (GaussianFluent)、4DGS + SLAM、4DGS + 端侧大模型的多模态融合

8.2 与本项目的关系

本项目 README 锁定的 Snapdragon 8 Gen 4 + Vulkan 1.3 实时渲染路径, 与上述 5 条方向高度交叉, 详见主文档 [01-/02-/03-]。

本节对应 **paper notes** 索引: [paper-notes/INDEX.md §G]

9 CONCLUSION

本综述系统性整理了 2023–2026 H1 移动端实时 4DGS 方向的 50+ 篇代表性工作, 覆盖表示 / 训练加速 / 渲染加速 / 移动端部署四大主题。当前 Snapdragon 8 Gen 4 平台已能实现桌面级 3DGS 实时渲染, 但 4DGS 仍受限于显存带宽 / Vulkan 驱动成熟度 / 功耗墙三重约束。

后续研究方向包括 feed-forward 4DGS、硬件-软件协同设计、4DGS ray tracing、时序压缩标准、跨模态融合等。本综述与项目 README 互补, 前者侧重学术调研纵深, 后者侧重工程落地路径。

本节对应 **paper notes** 索引: [paper-notes/INDEX.md]

REFERENCES

- [1] et al. Chen. 2025. 4DGS-CC: Contextual Coding Framework for 4D Gaussian Splatting Compression. (2025).
- [2] et al. Du. 2026. Flux-GS: High-Performance Neural Rendering on Snapdragon 8 Gen 3. In *ECCV*.
- [3] et al. Du. 2026. Mobile-GS: Real-Time Neural Radiance Field Rendering on Mobile GPUs. In *arXiv preprint arXiv:2603.11531*.
- [4] et al. Feng. 2025. Lumina: Real-Time Mobile Neural Rendering. (2025). SJTU + Rochester.
- [5] et al. Ghosh. 2026. GS-NFS: 4D Gaussian Splatting Decode on Jetson Orin. (2026). NVIDIA Research.
- [6] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. In *ACM Transactions on Graphics (SIGGRAPH)*.
- [7] et al. Lee. 2025. OMG4: Minimal 4D Gaussian Splatting via Imperceptible Temporal Pruning. (2025).
- [8] et al. Liu. 2025. 4DGRT: 4D Gaussian Splatting Ray Tracing. (2025). NTU + Intel.
- [9] et al. Oh. 2025. Neo: On-Device 3D Gaussian Splatting with Reuse-and-Update Sorting Accelerator. (2025).
- [10] et al. Song. 2026. L2D2-GS: Feedforward 4D Gaussian Splatting with Self-Supervised Densification. In *ECCV*. Xiaomi + PKU.
- [11] et al. Wang. 2026. RetimeGS: Continuous-Time 4D Gaussian Splatting for Dynamic Scene Rendering. In *CVPR*. Oral, HKUST + Netflix.
- [12] Guanjun Wu, Taoran Yi, Jiemin Fang, Leonidas Guibas, Ping Tan, Dahua Lin, and Gordon Wetzstein. 2024. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. In *arXiv preprint arXiv:2310.08528*.
- [13] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Chen, and Xiaogang Wang. 2023. Deformable 3D Gaussians for High-Fidelity Monocular Dynamic Scene Reconstruction. In *arXiv preprint arXiv:2309.13101*.
- [14] et al. Yuan. 2025. 4DGS-1K: High-Quality 4D Gaussian Splatting Dataset and Comprehensive Evaluation. (2025).
- [15] et al. Zhang. 2024. MEGA: Memory-Efficient 4D Gaussian Splatting with Buffer-A/B Residual Coding. (2024).